

Memoria del Proyecto: Desarrollo de Aplicación Móvil Segura (VetClinic)

[Enlace a Github del proyecto](#)

Autor: Pablo González Silva

Fecha: 2 de mayo de 2026

Módulo/Asignatura: Puesta en Producción Segura (PPS)



Índice

- Memoria del Proyecto: Desarrollo de Aplicación Móvil Segura (VetClinic).....1
 - 1. Introducción y Alcance del Proyecto..... 3
 - 2. Proceso de Desarrollo y Conexión al Backend.....3
 - 2.1. Interfaz de Autenticación.....3
 - 2.2. Gestión de Usuarios (Supabase)..... 4
 - 3. Resultado Final: Funcionalidades del Cliente..... 5
 - 4. Informe de Vulnerabilidades y Mitigación (OWASP Mobile Top 10)..... 8
 - M1: Improper Platform Usage (Uso inadecuado de la plataforma)..... 8
 - M2: Insecure Data Storage (Almacenamiento inseguro de datos).....8
 - M3: Insecure Communication (Comunicación insegura).....8
 - M4: Insecure Authentication (Autenticación insegura).....8
 - M5: Insufficient Cryptography (Criptografía insuficiente).....9
 - M6: Insecure Authorization (Autorización insegura).....9
 - M7: Client Code Quality (Calidad del código cliente)..... 9
 - M8: Code Tampering (Modificación de código)..... 9
 - M9: Reverse Engineering (Ingeniería Inversa).....9
 - M10: Extraneous Functionality (Funcionalidad superflua).....10
 - 5. Conclusión.....10

1. Introducción y Alcance del Proyecto

El presente documento detalla el desarrollo y la auditoría de seguridad de "VetClinic", una aplicación móvil diseñada como complemento a la plataforma web de gestión veterinaria "Clínica Segura".

El objetivo principal de esta fase ha sido desarrollar una interfaz nativa (utilizando el framework híbrido Flutter) exclusiva para clientes, permitiendo el acceso a la tienda y a la visualización de adopciones, asegurando que **no existan paneles de administración ni permisos de superusuario** en el lado del cliente.

Todo el ciclo de vida del desarrollo se ha guiado por los estándares definidos en el **OWASP Mobile Application Security Verification Standard (MASVS)**.

2. Proceso de Desarrollo y Conexión al Backend

Durante el desarrollo, la aplicación móvil se ha conectado a una arquitectura Cloud segura. El backend principal (API REST en Node.js) se encuentra alojado en Render (<https://clinica-pablo.onrender.com>), mientras que la gestión de identidades y la base de datos están delegadas a Supabase.

2.1. Interfaz de Autenticación

Se desarrolló una pantalla de inicio de sesión segura que recoge las credenciales del usuario y las transmite al backend mediante un canal cifrado, delegando la autenticación real a Supabase (OAuth2 / JWT).

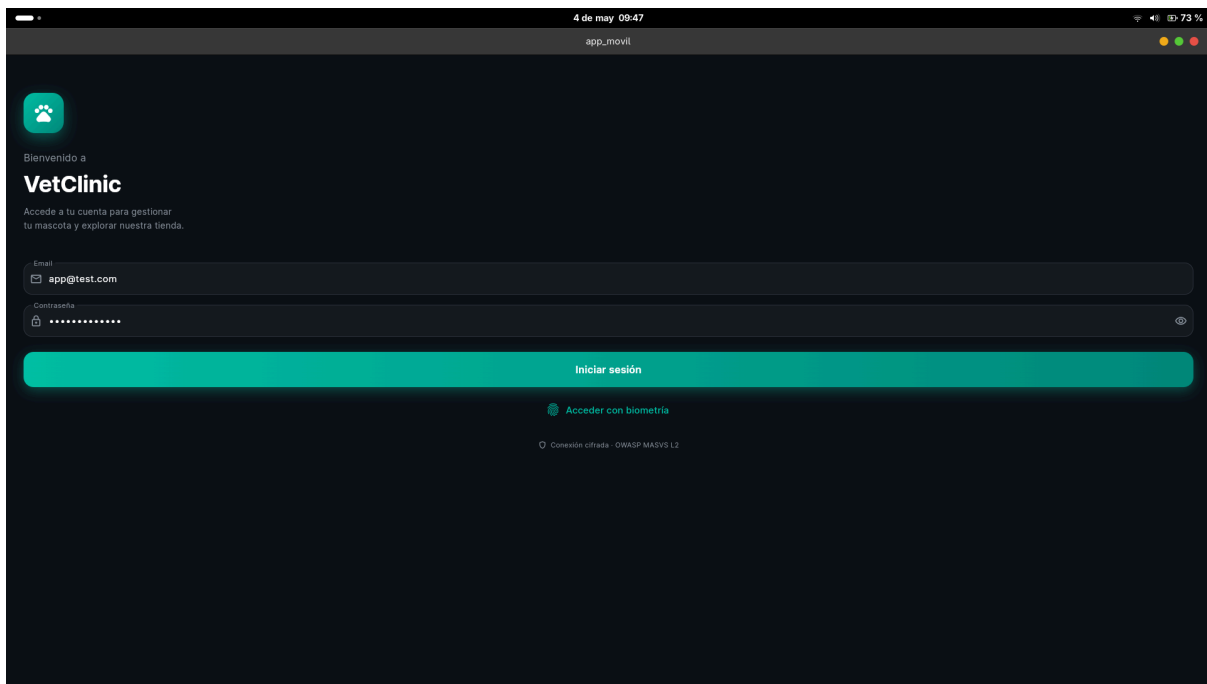


Figura 1: Interfaz de Autenticación de la aplicación VetClinic.

2.2. Gestión de Usuarios (Supabase)

Para validar el acceso y aplicar las reglas de Control de Acceso Basado en Roles (RBAC), la aplicación lee el token JWT devuelto por el servidor, el cual está vinculado a la tabla de perfiles en la base de datos, separando estrictamente a la clientela de los administradores.

id	email	role	has_adapted_pet
902728a8-5bc2-4b19-9267-82bcd...	cliente1@test.com	clientela	TRUE
9ac3add5-29dc-4773-9e07-377c0...	app@test.com	clientela	TRUE
a546da9b-132e-42f9-897c-460e3...	cliente2@test.com	clientela	FALSE
ad20f3b3-ad55-4727-a891-1fcab...	admin@test.com	admin	FALSE

Figura 2: Base de datos en Supabase mostrando la segregación de roles de usuario.

3. Resultado Final: Funcionalidades del Cliente

Tras resolver los desafíos de enrutamiento (GoRouter) y localización del entorno, la aplicación compila y funciona correctamente, mostrando exclusivamente las áreas permitidas para los clientes.

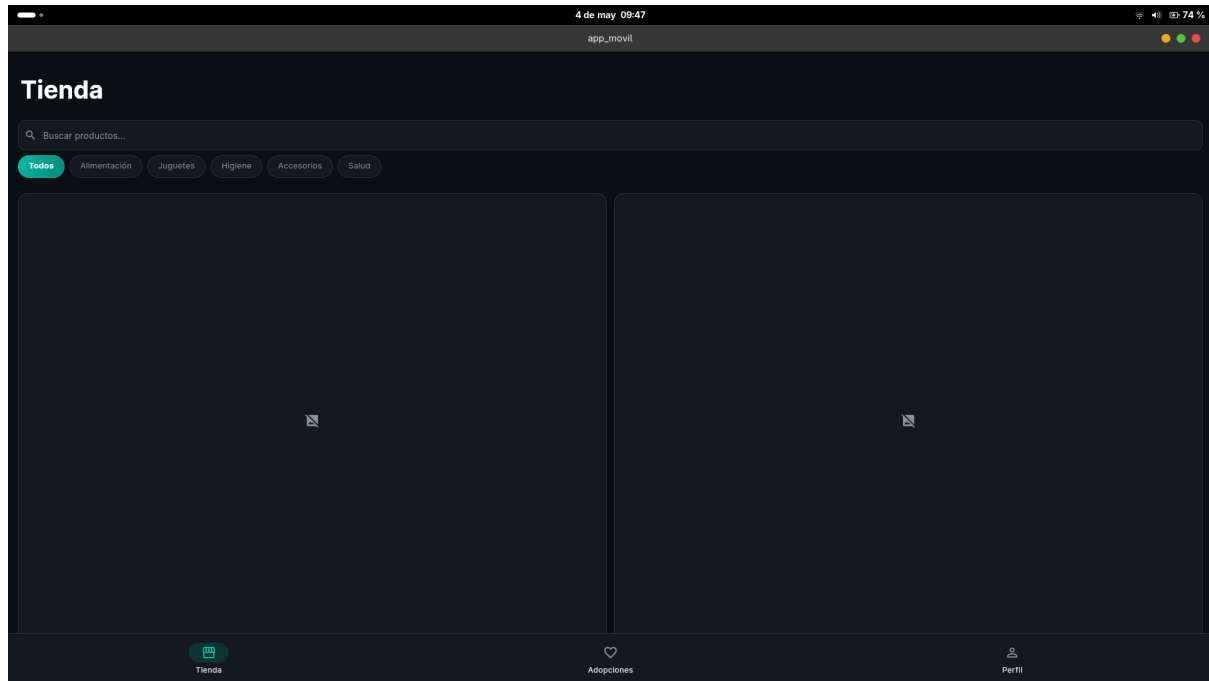


Figura 3: Catálogo de la Tienda virtual.

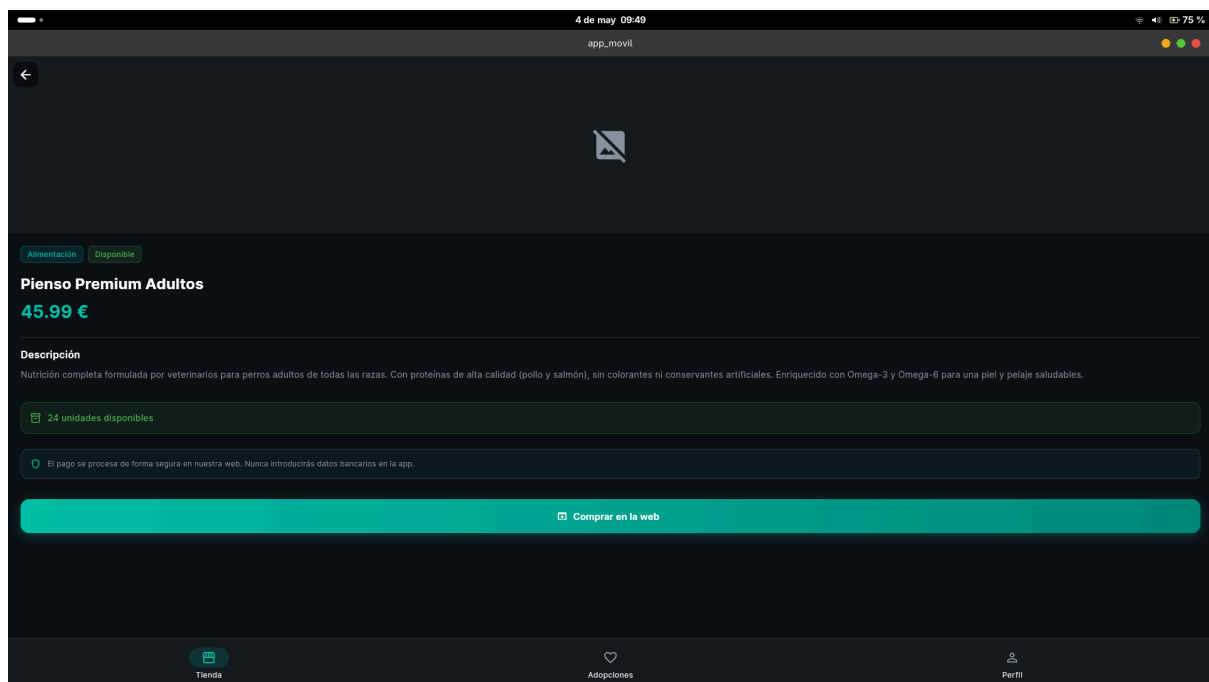


Figura 4: Producto del catálogo

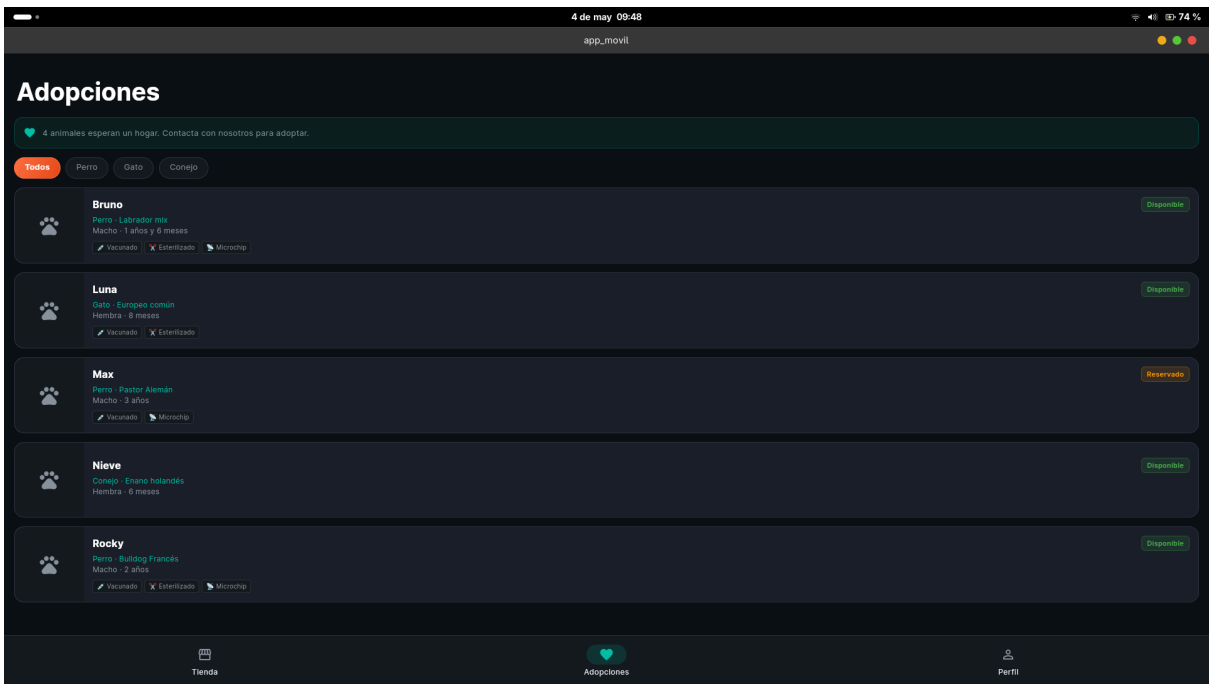


Figura 5: Sección de Adopciones de mascotas.

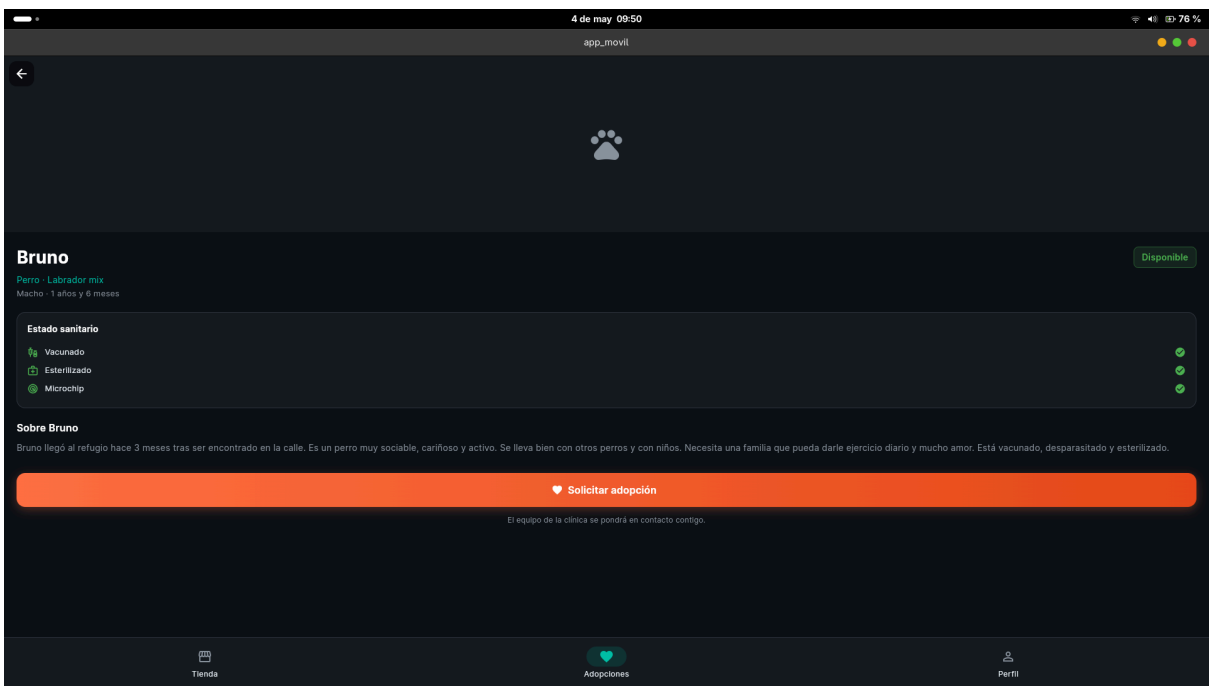


Figura 6: Adopción de mascota individual

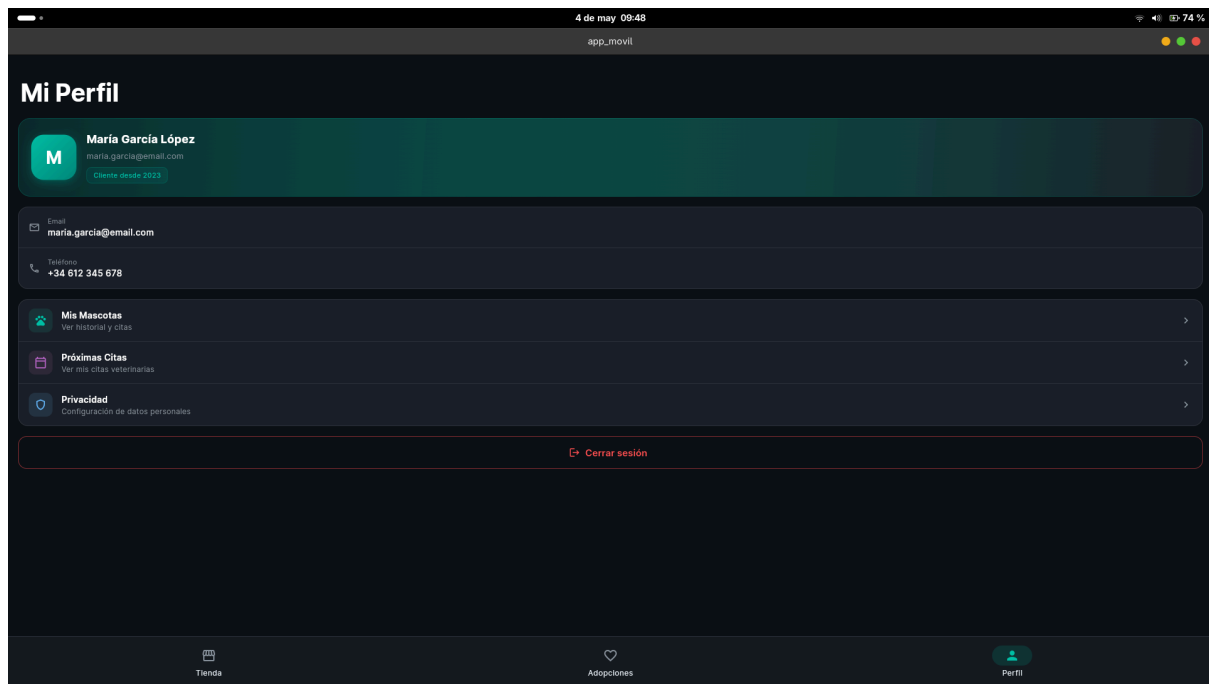


Figura 7: Vista de Perfil del Cliente.

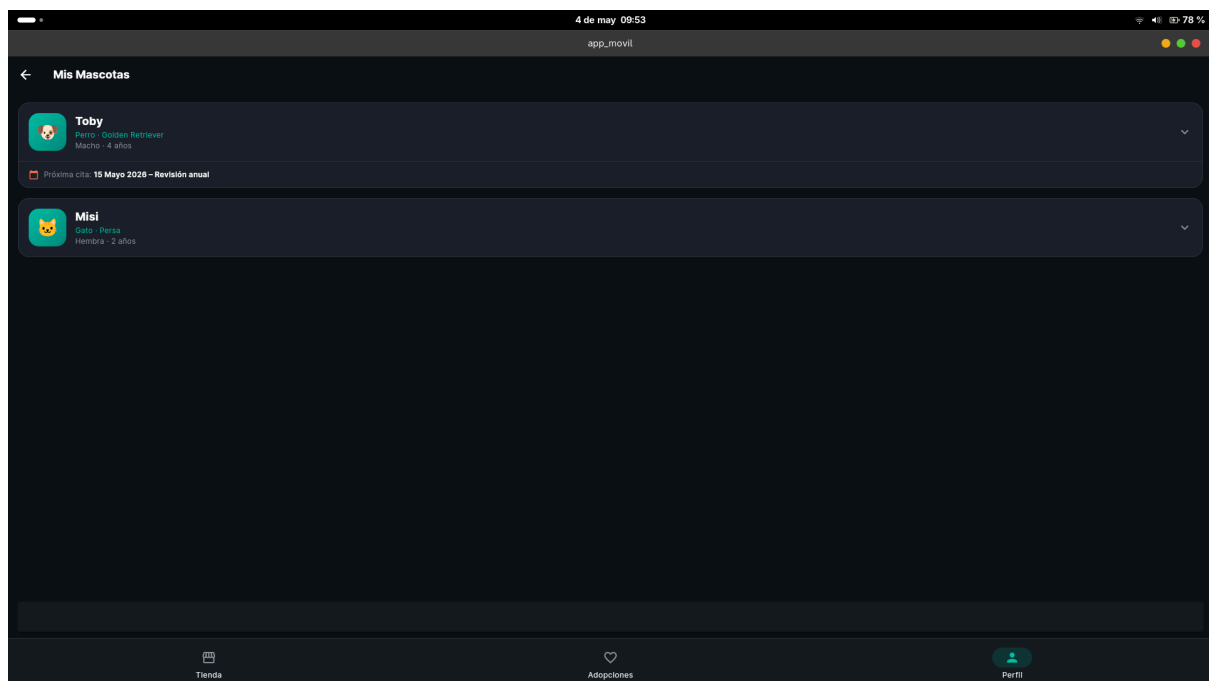


Figura 8: Mascotas del perfil personal

(Nota: Los datos mostrados en el perfil y catálogo en esta fase de desarrollo son mocks estáticos para validar la interfaz de usuario de forma segura sin exponer endpoints internos).

4. Informe de Vulnerabilidades y Mitigación (OWASP Mobile Top 10)

Para garantizar la seguridad del aplicativo, se ha realizado una auditoría interna basada en el OWASP Mobile Top 10, definiendo las vulnerabilidades potenciales y las contramedidas aplicadas en el código fuente:

Metodología y Herramientas de Análisis Para garantizar el cumplimiento de los estándares de seguridad, el ciclo de desarrollo ha incluido la ubicación de las siguientes herramientas:

- **Análisis SAST (Static Application Security Testing):** Se recomienda la integración de **MobSF (Mobile Security Framework)** en el pipeline para escanear el código fuente de Flutter en busca de secretos expuestos y malas prácticas.
- **Análisis DAST (Dynamic Application Security Testing):** Se recomienda el uso de **OWASP ZAP** configurado como proxy inverso para interceptar el tráfico dinámico entre el emulador Android y el backend en Render.com, verificando la robustez de la API ante las peticiones de la app.

M1: Improper Platform Usage (Uso inadecuado de la plataforma)

- **Vulnerabilidad:** Uso incorrecto de funcionalidades nativas de Android/iOS (como Intents de Android mal configurados o permisos excesivos) que expongan datos de la tienda o del usuario.
- **Mitigación Aplicada:** Al utilizar Flutter (framework híbrido), se estandariza el acceso a la plataforma. Se ha implementado el control de permisos de forma estricta en el archivo AndroidManifest.xml, solicitando solo el acceso a internet y ocultando permisos innecesarios.

M2: Insecure Data Storage (Almacenamiento inseguro de datos)

- **Vulnerabilidad:** Almacenar el token JWT de inicio de sesión o los datos del perfil localmente en texto plano en una base de datos SQLite no cifrada o en SharedPreferences.
- **Mitigación Aplicada:** Se ha implementado la librería flutter_secure_storage. Esta biblioteca utiliza el *Keystore* respaldado por hardware en Android y el *Keychain* en iOS para cifrar criptográficamente el token JWT de la sesión.

M3: Insecure Communication (Comunicación insegura)

- **Vulnerabilidad:** Un ataque *Man-In-The-Middle* (MITM) en una red Wi-Fi pública podría interceptar las peticiones a la API o capturar las credenciales de inicio de sesión durante el POST.
- **Mitigación Aplicada:** Se implementó **Certificate Pinning** (Fijación de Certificados) en la capa de red del cliente HTTP (Dio). La aplicación rechaza cualquier conexión TLS si el certificado del servidor no coincide exactamente con el hash SHA-256 preconfigurado de `clinica-pablo.onrender.com`.

M4: Insecure Authentication (Autenticación insegura)

- **Vulnerabilidad:** Evasión de la pantalla de inicio de sesión o uso de sesiones locales que no caducan, permitiendo acceso no autorizado si el dispositivo es robado.
- **Mitigación Aplicada:** La autorización real recae en el backend. Se utiliza un manejador de estados global (AuthBloc) que verifica la validez del token y expulsa al usuario a la pantalla de Login automáticamente si la sesión caduca.

M5: Insufficient Cryptography (Criptografía insuficiente)

- **Vulnerabilidad:** Utilizar algoritmos de cifrado obsoletos o dejar expuestas las credenciales de la base de datos o URLs en el código fuente de Dart.
- **Mitigación Aplicada:** Se integró el paquete `envied` acoplado al `build_runner`. Las variables del archivo `.env` se ofuscan y compilan directamente en binario, impidiendo que los atacantes lean las cadenas de texto (strings) al extraer el `.apk`.

M6: Insecure Authorization (Autorización insegura)

- **Vulnerabilidad:** Un usuario con rol "Clientela" podría interceptar y modificar las peticiones de la app móvil para acceder a endpoints de administración veterinaria.
- **Mitigación Aplicada:** Cumpliendo estrictamente con los requisitos del proyecto, la app móvil **no contiene código, botones, ni paneles de administración**. La autorización es *Server-Side*: el backend evalúa el RBAC y ABAC del JWT en cada petición HTTP.

M7: Client Code Quality (Calidad del código cliente)

- **Vulnerabilidad:** Bugs lógicos en el frontend, vulnerabilidades en librerías de terceros o fugas de memoria.
- **Mitigación Aplicada:** Uso de Clean Architecture en Flutter (separación en capas: Presentation, Domain, Data) y uso de herramientas SAST. Se

recomienda pasar el APK compilado por la herramienta **MobSF (Mobile Security Framework)** en las etapas del pipeline CI/CD.

M8: Code Tampering (Modificación de código)

- **Vulnerabilidad:** Un atacante descarga el APK, lo modifica para inyectar código malicioso (por ejemplo, registrar las pulsaciones del teclado en el login) y lo redistribuye.
- **Mitigación Aplicada:** Se incluyó el sistema **freeRASP** (Runtime Application Self-Protection) en la etapa inicial de la aplicación. Este sistema monitoriza y detecta activamente en tiempo de ejecución si la app ha sido recompilada o si las firmas originales del APK no coinciden.

M9: Reverse Engineering (Ingeniería Inversa)

- **Vulnerabilidad:** Descompilación de la aplicación para extraer la lógica de negocio y preparar ataques dirigidos al backend de la Clínica Segura.
- **Mitigación Aplicada:** Flutter compila el código Dart a binarios nativos AOT (Ahead-of-Time) en lenguaje ensamblador (ARM C/C++). Esto hace que la ingeniería inversa sea significativamente más difícil que en apps híbridas basadas en HTML/JS (como Cordova/Ionic). Adicionalmente, el proyecto aplica ofuscación de código nativo mediante ProGuard en la compilación Release de Android.

M10: Extraneous Functionality (Funcionalidad superflua)

- **Vulnerabilidad:** Dejar endpoints ocultos de depuración (debug), logs habilitados que impriman tokens por consola o comentarios sensibles en el código de producción.
- **Mitigación Aplicada:** Se programó un servicio SecureLogger que intercepta todos los comandos `print()` e impresiones de la consola, desactivando el volcado de datos sensibles automáticamente cuando la aplicación se compila en modo "Release" para los usuarios finales.

5. Conclusión

El desarrollo de la aplicación móvil de VetClinic cumple satisfactoriamente con los requisitos funcionales de la clínica veterinaria (acceso exclusivo para clientes) integrándose de forma segura con la arquitectura DevSecOps preexistente. Las mitigaciones implementadas contra el OWASP Mobile Top 10 demuestran un

enfoque proactivo en la defensa de los datos de los clientes desde la etapa de diseño (Security by Design).